

Feasibility Study – Neural-Network Stranger Pairing

DATE: 03-OCT-2026 STATUS: DRAFT FOR REVIEW – research only, no code has been written
RESPONDS TO: [2026-10-03a-neural-net-based-stranger-pairing.md] (2026-10-03a-neural-net-based-stranger-pairing.md) RELATED: [2026-10-03-combat-revision-technical-spec.md] (2026-10-03-combat-revision-technical-spec.md) (§4.5 legality, §5 strategies, §5.3 exact evaluator, §5.6 external strategy, §8 test harness)

This document assumes you understand in general how neural networks work, but have never trained one, exported one, or run one inside an application. Section 3 is a primer for exactly that gap; everything else builds on it. Where I rely on current facts about tools or Convex, I checked the vendor documentation and link it in the Sources section. Where I give numbers that I could not check, they are marked **estimate**.

1. Summary

Is it feasible? Yes. It is a small, well-defined learning problem, the open-source tooling is mature and free, and a MacBook is enough to train it. Three findings change what you should actually build, though.

1.1 Four findings that matter more than any tool choice

- 1. You may not need a neural network to get the best pairing — but a network is still worth building.** The strangers' pairing problem is small (a handful of creatures per side), and with your initial objective (§4.3a) every candidate can be scored *exactly* by simple addition — no dice at all (the dice-aware version in technical spec §5.3 is a later refinement). A search over legal formations (the expert strategy, spec §5.4) can therefore find the optimal pairing with no learning at all — for typical fights. A network earns its place for different reasons: **speed** on large fights where search is too slow, **imitating** a style (yours, or a tiered “difficulty personality”), and — as a stretch — **looking ahead across rounds**. The honest plan is: build the exact search first, then train the network to *imitate it* and measure how close it gets.
- 2. You probably don't need a Linux service.** A pairing model of this size is tens of thousands of numbers. You can export those numbers into a file in the repository and run the model as ~50 lines of TypeScript **inside the Convex mutation itself** — no network call, no server to run, no new failure mode, and perfectly repeatable. Convex gives mutations 1 second of user-code time; a small model uses a fraction of that (**estimate**, to be measured in Stage 3). The Linux-service design you suggested is the right answer only if the model later grows beyond what fits in the mutation (a transformer, a reinforcement-learning policy).
- 3. The hard part is not the model; it is the data and the contract.** Training data must be generated by the game's own engine, features must be computed by the *same* code at training time and at play time, and the objective (“best” for whom, measured how?) must be defined first. That objective is still an open item in the requirements (technical spec G1).
- 4. Your initial objective needs one refinement before it can teach a network anything.** Maximising “stranger strength vs player strength” *summed over the matches* gives the **same number for every way of pairing ordinary fighters**: the sum only depends on *which creatures are engaged*, not on *who fights whom*. So it cannot tell a clever pairing from a poor one. Example (engine strengths): strangers Ogre 5 and Dwarf 1 against a Hero with the Magic Sword (7) and a Man (3) score **−4**

either way, yet one pairing leads in one match and the other leads in none (§4.3a). The fix is small — score each match on whether the strangers *lead*, not just by how much — and the options are listed there (N12).

1.2 Recommended path

Stage	What	Needs	Decision gate
0	Prerequisites: rules sign-off, round engine, legality layer, exact evaluator (tech spec M0–M2)	Peter's answers	Don't train on a moving target
1	No-ML baseline : exact/branch-and-bound expert strategy and the test harness	Stage 0	If expert is optimal and fast enough, the network becomes optional
2	Prototype on the MacBook: generate data, train two small models, measure how close they get to expert	PyTorch, scikit-learn	Stop if the model can't get within a few percent of expert's value
3	Ship it embedded in TypeScript as strategy nn-v1, behind the existing fallback	Stage 2	Parity tests pass; bench shows it beats standard
4	<i>Only if needed</i> : service on Linux, or reinforcement learning	Stage 3 learnings	A concrete limit was hit

Rough effort for someone new to ML, working part-time (**estimate**, to be recalibrated after Stage 1): Stage 1 1–2 weeks, Stage 2 2–4 weeks, Stage 3 1–2 weeks, Stage 4 3–8+ weeks. Compute cost for Stages 1–3 is **nothing beyond your own MacBook**.

1.3 Decisions I need from you

Your correction (the fourth input is the player's wielded artefacts) closes N3, and your initial objective answers part of N1. The decisions that remain are listed in §12; the ones that shape everything are **how to score a pairing** (N12 — see finding 4), **whether the network also decides who wields the strangers' artefacts** (N2 — your input list implies it does), and **whether the player's deployment is an input** (N11).

2. The problem, stated precisely

2.1 What you asked for

Choose the best pairing of strangers against the player's party in combat. The inputs:

- the stranger party members,
- the artefacts **available to the strangers** (not yet allocated),
- the player's party members,

- the artefacts **wielded by the player's party members** — the player has already decided how to distribute the party's artefacts among its creatures, and that is fixed for the fight (the party cannot redistribute during a fight).

Initial goal: **maximise points calculated as the stranger party's strength against the player party's strength**, with every pairing obeying the rules. (§4.3a looks closely at what that scoring does and does not distinguish.)

2.2 What the list implies, and what is still missing

- **The strangers' own loadout is part of the decision.** The strangers' artefacts are given as *available*, not as *wielded*, so something must decide who wields what. Either the default loadout (`strangers-default-loadout.md`) runs first and the network only pairs, or the network decides the loadout as well as the pairing. Your list reads as the second. That matters: the player's artefacts are visible, so a learned loadout can answer them (a Shield for the stranger who faces the player's caster, say) — which would make the strangers stronger than the default procedure (N2, N13). Section 4.6 covers how to build it either way.
- **The player's deployment.** In Peter's round structure the strangers pair against the *deployed front line* of the defender (each-round-is-fought, step c). So the input must say which of the player's creatures are in the front line, which are held back or in the background, and which are already engaged in a persisting match — not just who is in the party (N11).
- **Context.** Whether an Eye of God is present (it switches off artefacts), active curses, the dungeon level (the Ring makes its bearer invincible from level 4), the round number, who is attacking, and the surprise bonus (tech spec §4.1–§4.2).

2.3 What the network must produce

A **formation**: which stranger fronts which of the player's creatures (and where casters stand), always legal under the rules — and, if N2 is “yes”, a **loadout** chosen before it (who wields which available artefact). Legality is enforced by the engine (tech spec §4.5), not learned — the network only ever *ranks legal options*. That is the single most important design decision for safety, and it is what keeps a mistake in the model from ever producing an illegal fight.

2.4 How big is the search?

Number of one-to-one assignments of k strangers onto n of the player's creatures, ignoring two-against-one joins and background casters (so a lower bound):

Strangers	Party creatures	Assignments
2	3	6
3	4	24
4	6	360
5	8	6,720
6	10	151,200
8 (<i>the engine's chamber cap</i>)	14 (<i>a real game reached 14</i>)	121,080,960

Typical fights sit in the first rows, where exhaustive scoring is instant. The bottom rows are why a *learned* shortcut is attractive — and also why they are rare enough that a fast heuristic fallback is acceptable.

The engine encodes **21 creature types** and **22 treasure types**, of which eight matter in a fight (Sword, Staff, Ring, Axe, Shield, Eye of God, Talisman, Scroll). The inputs are therefore small and fixed in vocabulary.

3. A primer for this project (custom models from zero)

If you only skim one section, make it this one. It maps the general ideas you already know onto what you would actually *do*.

3.1 The life of a custom model

- | | |
|------------------------|---|
| 1. Define the task | → "given this fight, score a formation" |
| 2. Make data | → run the game engine millions of times, record (situation, formation, how good) |
| 3. Encode as numbers | → a "feature vector" per example (the same code later runs in the game) |
| 4. Split the data | → train (learn from) / validation (tune on) / test (final exam, touched once) |
| 5. Train | → a script nudges the model's numbers to reduce a "loss" (prediction error) |
| 6. Evaluate | → does it rank formations as well as the exact search? is it fast? is it always leg |
| 7. Export | → write the learned numbers to a file |
| 8. Run ("inference") | → the game loads the file and computes scores; nothing is learned at play time |
| 9. Monitor and retrain | → rules change → regenerate data → retrain → ship a new versioned file |

Training happens **once, offline, on your laptop**. What reaches the game is *just a file of numbers* plus a few lines of arithmetic. Nothing in the game “learns” while people play.

3.2 Words you will meet

- **Feature**: one input number (e.g. “this stranger’s fighting strength”, “the Eye is present”).
- **Label / target**: the answer you want the model to predict (here: how good a formation is).
- **Loss**: a single number saying how wrong the predictions are; training tries to shrink it.
- **Epoch**: one pass over the whole training set.
- **Overfitting**: the model memorises the training examples instead of learning the pattern; you detect it when validation error stops improving while training error keeps falling.
- **Parameters / weights**: the numbers the model learns. Ours: roughly 10^4 – 10^5 — tiny by modern standards.
- **Inference**: using the finished model (no learning).
- **ONNX**: a standard file format for trained models, readable by many languages.

3.3 What “training on a MacBook” physically looks like

A project folder, a Python virtual environment, a script of ~50 lines that loads a data file and loops, and a few minutes of the laptop’s fans. You watch a loss number fall in the terminal (or a graph in TensorBoard) and a file appears at the end. It is closer to running a build than to anything exotic. Appendix A has a complete sketch.

4. How to frame the problem for a network

4.1 Representing the situation

A fight is two **sets** of creatures with equipment. Two workable encodings:

- **Fixed slots (recommended for v1).** Reserve, say, 8 stranger slots and 14 party slots; each slot holds a creature-type code plus its strength, magic, borne-artefact flags and kill count; empty slots are zeros. Add context features (Eye, curses, level, round, attacker, surprise). Simple, works with any model type, and the bounded sizes above make it practical. Downside: it treats the order of creatures as meaningful, so the training data should be *canonicalised* (sorted) and optionally shuffled to teach order doesn't matter.
- **Set encoders** (Deep Sets, attention/transformer). Order-free by construction and handles any size. More powerful, more to learn, and heavier to embed. A later upgrade, not a first step.

4.2 What the network outputs – four options

1. **Scorer over enumerated legal formations (recommended).** Input = situation + one candidate formation; output = a single score (predicted expected value). At play time the engine's legality layer lists the legal formations, the model scores them all, and the best wins. Strengths: legality is guaranteed; the model is small; training is ordinary regression; works naturally for every decision hook (deploy, engage, redeploy). Weakness: you must be able to enumerate candidates — fine for the common rows of §2.4, needs pre-filtering for the big ones (score only the top K from a quick heuristic).
2. **Imitation classifier.** Input = situation; output = “which candidate is best”. Awkward because the set of candidates changes size every fight.
3. **Step-by-step policy (pointer network style).** The model builds the pairing one assignment at a time with illegal choices masked out. Elegant for large fights; considerably harder to build, train and embed.
4. **Reinforcement learning (AlphaZero-style self-play).** The model learns by playing whole fights against itself. Can discover multi-round tactics nobody wrote down; needs much more compute and engineering (§5.3).

Recommendation: option 1 first. It matches your “pairing” goal, reuses the legality layer, and trains in minutes.

4.3 Where the training labels come from

This is the reason the project is *easier* than most ML projects: **there is a perfect teacher**. With the initial objective the score of any candidate is plain arithmetic on the engine's own strength calculation (Peter's steps i and ii) — **no randomness in the label at all**. (If the objective later includes the dice, the exact evaluator in tech spec §5.3 computes the expectation by enumerating the 36 outcomes per match; the labels are still exact.) So:

- Generate scenarios with the same harness that tests strategies (tech spec §8): random parties with their wielded artefacts, strangers with their available artefacts, weighted towards realistic sizes.
- For each, enumerate the legal candidates, label each with its exact score, and store (situation features, candidate features, score).
- Train the model to predict that score. No dice noise, no human labelling, no reinforcement learning.

Practicalities: split train/validation/test **by scenario** (never put two candidates from the same scenario on both sides of a split, or the exam leaks); keep a held-out set of *large* fights to see how well it generalises beyond the sizes it saw most; version the data with the seed and the engine git SHA. With a deterministic objective, exact **ties** are common, so the pipeline needs a fixed tie-break rule (below) or the labels will contradict themselves.

4.3a The initial objective, examined

As stated. For a candidate (a legal loadout plus a legal formation), let $S_{str}(m)$ and $S_{pty}(m)$ be the total effective strengths of the two sides in match m (Peter’s steps i–ii: positional effects, artefact effects — Eye, Axe, Shield, Staff, Sword, Ring, Talisman — and immaterial creatures; **no** die, surprise or Ring-bonus terms). The objective is

$$J = \sum \text{over matches } m \text{ of } (S_{str}(m) - S_{pty}(m)) \quad - \text{ maximise over legal candidates}$$

What it does well. It rewards good *loadouts*: giving the Staff to the stranger who gains most from it, the Axe to a Dwarf, the Shield to the creature facing the player’s caster — all change the totals. It also rewards the strangers engaging strongly and the player being forced to engage weakly.

What it cannot see. Because the sum splits into “all engaged stranger strength” minus “all engaged player strength”, for ordinary fighters **every pairing of the same engaged creatures scores identically**. Worked example, using the engine’s strengths (Ogre 5, Dwarf 1, Hero 5 + Magic Sword 2 = 7, Man 3):

Pairing	Margins	Sum of margins (J)	Matches the strangers lead
Ogre v Hero, Dwarf v Man	−2, −2	−4	0
Ogre v Man, Dwarf v Hero	+2, −6	−4	1

J is blind to the difference that matters: the second pairing wins a match and the first does not. Pairing only changes J where an effect is *local to a match* — the Shield cancelling the opposing side’s magic, a background caster supporting its own front line, the Spectre/Demon rules, or which creatures are engaged at all.

Options that keep your idea but make pairing matter (all deterministic, all computed from the same strength numbers):

- **R1 — matches led** (recommended first version). Count the matches where $S_{str}(m) > S_{pty}(m)$, with ties worth half. This is the closest “points” version of your goal and directly rewards pairings that win matches. Break ties by the total margin J , then deterministically (a fixed order, or a seeded choice recorded in the log).
- **R2 — capped margin.** $\sum \text{clamp}(S_{str}(m) - S_{pty}(m), -c, +c)$: rewards winning many matches comfortably without letting one huge mismatch dominate.
- **R3 — add the roll terms.** Include surprise $\{0,1,2\}$ and the Ring $\{0,1\}$ (they are added to the roll, not to strength) so “leads” matches what will actually be compared.
- **R4 — expected value with the dice.** Probability of winning each match and the value of the casualties, from exact enumeration (tech spec §5.3). The most faithful, and still exact.
- **R5 — across rounds.** Persisting matches and alternation (tech spec §4.2); only worth it if R1–R4 prove too short-sighted.

Rules always win. None of these may make an illegal pairing attractive: the candidate list comes from the engine’s legality layer (§2.3), so “maximise” only ever ranges over legal formations — including the *must engage* and *match as best you can* obligations (high-level requirement 3).

4.4 Which kind of model

For a first experiment train **two**, because the comparison teaches you the most:

- **Gradient-boosted trees** (LightGBM/XGBoost). Excellent on small, tabular, structured inputs like this; often matches or beats a network with far less tuning. Can be turned into plain JavaScript (§6).
- **A small MLP** (multi-layer perceptron: 2–3 hidden layers). The “neural network” you asked about. Simple, and trivial to run as hand-written TypeScript.

If the trees win, that is a perfectly good outcome — you wanted a pairing system, not necessarily a neural one.

4.5 How we will know it works

Measured by the test harness (tech spec §8), against the exact expert strategy and the default standard:

- **Regret:** average shortfall in the chosen objective (§4.3a) versus the optimal candidate (target: small, say within a few percent — to be set after Stage 1).
- **Top-1 agreement:** how often it picks the optimal formation.
- **Arena win rate:** whole fights played to the end against reference party policies.
- **Legality:** 100%, by construction (the engine only offers legal candidates).
- **Latency:** within the budget of wherever it runs (§6).

4.6 If the network also chooses who wields what

Your input list implies the network decides the strangers’ loadout. The loadout is a separate, smaller decision made **before** pairing (the *Stranger Fight Preparations* step of the combat overview), and it can be learned the same way:

- **Candidates.** Each available artefact goes to one *eligible* creature (Peter’s who-can-use matrix) or stays on the floor. Upper bound with a artefacts and m eligible bearers: $(m + 1)^a$, e.g. 6 bearers and 5 artefacts = 16,807 — and eligibility cuts that sharply in practice. Cheap to enumerate.
- **Labels.** For each loadout candidate, the score of the **best pairing** available with it (the inner search from Stage 1). The model learns “how good is this loadout”, not a pairing.
- **Two small models, not one big one:** a loadout scorer and a pairing scorer, sharing the same encoder.
- **Baseline to beat:** the default loadout. A learned loadout is only worth shipping if it scores better on the objective **and** you are happy for the strangers to react to the player’s artefacts (N13). The default procedure ignores them, so a learned one can be a harder opponent — a natural way to define a higher difficulty level.
- **Rules still apply.** Artefacts must go to creatures that can bear them (the matrix), the Eye switches the procedure off, and one creature may bear several artefacts (Axe and Sword are not used together in one fight).

5. The open-source landscape

5.1 Training frameworks

- **PyTorch** (*recommended*). The standard research and hobbyist framework; vast tutorials; first-class ONNX export; runs on Apple-silicon GPUs through the “MPS” backend, available since PyTorch 1.12.

Known limits worth knowing: some operations and optimiser features lag behind NVIDIA support, and on macOS a data loader with `num_workers > 0` often *slows* training — keep it at 0 for a small model like this.

- **scikit-learn, LightGBM, XGBoost.** For classical models (trees, linear models). Trivial to run on a laptop CPU.
- **MLX** (Apple, MIT licence). An array and neural-network library designed for Apple silicon’s unified memory, with an API close to PyTorch and NumPy. Attractive for large models on a Mac, but PyTorch’s ecosystem and ONNX export are easier for a first project, and this model is too small for the speed difference to matter.
- **JAX** and **TensorFlow/Keras.** Both capable; neither offers an advantage for this problem, and Apple-GPU support is less smooth than PyTorch’s. Skip.

5.2 Workflow tools (keep this minimal)

- **uv** or **conda** for a reproducible Python environment; **Jupyter** for exploring data.
- **TensorBoard** (free, local) to watch training curves. **MLflow** (local) if you want to record runs; Weights & Biases is a hosted alternative — optional.
- A plain folder convention plus a `data/` hash is enough versioning at this scale; heavier tools (DVC) can wait.

5.3 Game-AI and reinforcement-learning frameworks (only if Stage 4 happens)

- **OpenSpiel** (Google DeepMind) is a framework for reinforcement learning in games, with a Python AlphaZero implementation. Its documentation notes the Python version runs actors as separate processes, does not batch inference, and uses the CPU; a C++ version supports GPUs and batching. Adding the Sorcerer’s Cave round engine as a custom OpenSpiel game is a real project in its own right.
- **Gymnasium / PettingZoo, Stable-Baselines3, Ray RLlib** are the usual environment and algorithm libraries.

Verdict: overkill for pairing a single round, valuable only if you later want multi-round tactics learned from self-play. Not part of the recommended path.

5.4 Model formats and inference runtimes

- **ONNX**, the portable model format, exported from PyTorch with `torch.onnx.export`. Runtimes exist for Python, Node (`onnxruntime-node`, native) and browsers (`onnxruntime-web`, WebAssembly).
 - **Plain weight files** (JSON or binary) for a model small enough to execute by hand. This is the option that needs no runtime at all.
 - **m2cgen** can turn a trained scikit-learn, XGBoost or LightGBM *tree* model into dependency-free JavaScript source. It does not support neural networks. Check that the project is still maintained before depending on it.
-

6. Getting a model into the game

6.1 What Convex allows (checked against the vendor documentation)

Fact	Consequence
Queries and mutations run only in Convex's default runtime and may use at most 1 second of user code	A model run inside the move must be fast
Default runtime memory 64 MiB ; Node-action memory 512 MiB	Rules out heavyweight runtimes in a mutation
The default runtime supports WebAssembly , but no native modules ; compiled modules count against bundle size	<code>onnxruntime-web</code> is technically possible
Code size limit 32 MiB per deployment	A 10–20 MB WebAssembly runtime is a poor fit
Node actions ("use node") can use native packages via "external packages", installed from npm on deploy; bundle plus packages $\leq$ 45 MB compressed / 240 MB uncompressed ; the docs do not state the platform or CPU architecture	<code>onnxruntime-node</code> might work — needs a spike
Actions can call out with <code>fetch</code> ; mutations cannot	Any HTTP service is called from an <i>action</i> (tech spec §5.6)
Node actions: up to 10 minutes	Plenty for a service call

6.2 The options compared

Option	Needs a service?	Model size it suits	Repeatable?	Ops burden
A. Embedded in TypeScript (weights file + hand-written forward pass)	No	Up to roughly 10^5 parameters (estimate)	Yes	None
B. Trees compiled to JavaScript (m2cgen)	No	Any tree model that fits the code limit	Yes	None
C. <code>onnxruntime-node</code> in a Convex Node action	No external server	Any ONNX model	Yes	Low, but unproven on Convex
D. <code>onnxruntime-web</code> (WebAssembly) in the default runtime	No	Small–medium	Yes	Low, but a 10–20 MB runtime vs 64 MiB RAM / 32 MiB code
E. HTTP service on your Linux machine (your idea)	Yes	Anything	Via logging (§6.4)	Highest: TLS, auth, monitoring, patching, uptime

6.3 Option A in detail (my recommendation)

- Export each layer's weights and biases to a JSON (or binary) file during training.
- Implement the forward pass in TypeScript: for each layer, multiply, add the bias, apply the activation (ReLU). A 3-layer MLP is about fifty lines.
- Cost, as an **estimate** to be measured in Stage 3: a $\sim 47,000$ -parameter model costs $\sim 47,000$ multiply-adds per candidate; scoring 1,000 candidates is ~ 47 million, which a JavaScript engine should do in roughly 0.05–0.25 s — inside the 1 s budget but not by a huge margin. Mitigations: pre-filter candidates to the top K by a cheap heuristic, or shrink the model.
- **Parity tests are mandatory:** the TypeScript forward pass must reproduce PyTorch's scores to $\sim 1e-5$ on a set of golden inputs. This is the usual place for bugs.
- **Keep it server-side.** The `@sorcerers-cave/engine` package is imported by the browser app. A strategy placed in it would ship the weights (your model, effectively) and the code to every player. Put the NN strategy in a server-only module under `convex/`.

6.4 Why the architecture already supports any of these

The technical spec records every stranger decision as an ordinary logged action (§5.2) and replay never runs a strategy. So **where** the model runs — in the mutation, a Node action, or your Linux box — makes no difference to replay, to bug reports, or to the engine. Each decision is also validated by the legality layer, and an illegal or late answer falls back to standard. The model is therefore strictly a *suggestion engine*; it can be wrong but not harmful.

6.5 If you do want the Linux service (option E)

It is a normal, well-trodden pattern:

- A small Python web app (FastAPI or similar) loading the exported ONNX model with ONNX Runtime — no GPU needed at this size.
- A reverse proxy for HTTPS, an API key checked on every request, rate limiting, and basic monitoring.
- Packaged in a container; the Convex action calls it with `fetch`, with a short timeout and the standard fallback.
- Add: certificates renewal, OS updates, backups of the model files, an uptime alert, and a plan for the box being down mid-game.

This is more to run than the model itself. It becomes the right choice if the model outgrows the mutation (a transformer or policy network) or you want to retrain and swap models without redeploying the game.

7. Training on a MacBook

- **Hardware.** Any Apple-silicon Mac. The first experiments (tabular data, $\sim 10^5$ -parameter model) run in minutes on the CPU; the Apple GPU (MPS) only starts to matter for bigger models.
- **Memory.** Unified memory is shared by CPU and GPU; a few hundred thousand to a few million training rows is comfortable. Store features as `float32` (NumPy `.npz` or Parquet).
- **Pitfalls specific to macOS/PyTorch:** keep `num_workers=0` in data loaders; make tensors contiguous before using optimiser operations that expect it; if an MPS operation is unsupported, set the CPU fallback environment variable or train on CPU — correctness first.

- **Self-play reinforcement learning** (Stage 4) is a different scale: hours to days of compute and a custom environment. Consider that a separate decision, and consider renting a GPU for it rather than leaving a laptop running.
 - **Reproducibility.** Fix random seeds; record the engine git SHA, the feature version and a hash of the dataset next to every saved model.
-

8. Data and the cross-language contract

The engine is TypeScript; training is Python. The only safe way to join them:

1. **One feature encoder, in TypeScript.** The same function turns a situation and a formation into numbers when exporting training data and when scoring in the game. Never re-implement it in Python.
 2. **A data export command** (a script in the harness) writing rows plus a `featureVersion` and a schema description. Python reads them; it never reconstructs the game.
 3. **Golden vectors.** A small fixed set of inputs with the expected scores, checked in CI against the TypeScript forward pass — this guards both the encoder and the exported weights.
 4. **A versioned model file** (`pairing_v1.json` with `featureVersion`, parameter count, training date, data hash, metrics). The game refuses to load a model whose `featureVersion` it doesn't know.
 5. **Rules change → retrain.** Any change to the artefact table or strength rules can invalidate a model. The pipeline must be one command so retraining is routine, and old models are marked superseded.
-

9. Where it fits in the technical spec

- It is one more `StrangerStrategy` (tech spec §5.1) with id `nn-v1`. If it runs embedded it is kind: `"builtin"`, not `"external"`; the external-service path in §5.6 then becomes optional.
 - It uses the same hooks: `assignArtefacts` (in scope if N2 is “yes”; §4.6), `deploy`, `engage`, `redploy`. Its inputs include the player's deployment (N11), and its output is checked against the legality layer.
 - The bench (§8) measures it against `standard` and `expert`, so “does the network help?” has an evidence-based answer, and the calibration of difficulty levels (G1, G10) uses the same numbers.
 - The legality layer (§4.5) supplies the candidates and is the final gate.
 - Each logged decision records the model version, so a bug report shows exactly which model made a choice, and a bad model can be switched off per game.
 - It cannot explain itself the way a rule can. The why trace (improvement I4) would carry the top few scores instead — enough to debug, not a narrative.
-

10. Risks

- **The network is no better than search.** Likely for typical fights. Mitigation: Stage 1 builds the search first and the gate in §1.2 stops the project honestly if there's nothing to gain.
- **A vague objective gives a vague model.** “Best” must be defined (G1) before labels exist; changing it later means regenerating everything.
- **Train/serve skew.** The commonest cause of “it worked in the notebook”. Mitigated by one TypeScript encoder and golden vectors (§8).
- **Distribution shift.** Real fights differ from generated ones (certain compositions are far more common). Mitigation: weight generation by observed game data and keep a large-fight held-out set.

- **Rules drift.** Rules and data discrepancies are already known (tech spec D1–D3). Training on wrong numbers produces a confidently wrong model; reconcile them in M0 first.
 - **Floating-point differences** between PyTorch and TypeScript. Mitigated by parity tests; decisions are logged, so they can never change a replay.
 - **Operations burden** (option E only). Mitigated by choosing option A.
 - **Scope creep into reinforcement learning.** Real value possible, but treat it as a separate project with its own go/no-go.
 - **Licensing.** The tools named here are open source under permissive licences (PyTorch, ONNX Runtime and MLX are widely used under BSD/MIT-style terms; XGBoost is Apache-2.0; LightGBM is MIT) — confirm the current licence text before shipping anything derived from a tool.
-

11. Staged plan

Stage 0 – prerequisites (not ML)

Rules sign-off for the open questions that change legality and strength (technical spec register, ★ items), the round engine, the legality layer and the exact evaluator. Without them there is no stable target to learn.

Stage 1 – the no-ML baseline

Build expert (exact evaluation plus search) and the harness. Output: a report of how often expert is optimal, its latency by fight size, and the value gap between standard and expert. **Gate:** if expert is optimal and fast enough, the network is a nice-to-have (speed, style), not a necessity.

Stage 2 – prototype on the MacBook

Generate 10^5 – 10^6 scenarios; train LightGBM and an MLP; compare regret, top-1 agreement and latency on held-out scenarios, including large fights. Output: a short results note. **Gate:** within a few percent of expert on typical fights and no pathological failures.

Stage 3 – ship it embedded

Export weights; write the TypeScript forward pass and golden-vector tests; add nn-v1 behind the fallback; measure in the bench and in the real mutation. **Gate:** parity, latency inside budget, beats standard.

Stage 4 – only if needed

Service on Linux (§6.5) if the model outgrew the mutation; or a reinforcement-learning project (§5.3) if multi-round tactics matter enough.

12. Questions for you

Answered by your latest message: **N1** (partly — the initial goal is a strength-based score; see N12) and **N3** (the fourth input is the player's wielded artefacts, so it is in).

- **N1 — Strongest, human-like, or tiered?** (*Partly answered.*) The initial goal is to maximise the strangers' strength against the player's, within the rules. Still open: is the end aim the strongest possible opponent, a plausible human-like one, or a ladder of difficulties? My default: treat the maximum as the top level and derive weaker tiers from it.

- **N2 — Does the network also choose the strangers’ artefacts?** Your input list gives the strangers’ artefacts as *available* (not *wielded*), which implies yes. My default: yes, as a second small model (§4.6), benchmarked against the default loadout and shipped only if it beats it.
- **N11 — Is the player’s deployment an input?** The strangers pair against the player’s *deployed front line* (Peter’s step c), so the network needs to know which player creatures are deployed, held back, in the background or already engaged. My default: yes, as extra input features.
- **N12 — How is a pairing scored?** The summed strength margin scores every pairing of the same engaged creatures the same (finding 4, §4.3a). My default: **R1, matches led** (ties half, then total margin as the tie-break), moving to R3/R4 once the engine’s round flow is settled. Please confirm, or say which of R1–R5 you intended.
- **N13 — May the strangers’ loadout react to the player’s artefacts?** If yes, a learned loadout can be stronger than the default procedure. My default: allow it only at a harder difficulty level; the standard level keeps the default loadout.
- **N5 — Do you have recorded human decisions to learn from?** If you (or Peter) have played the strangers’ side, even informally, those choices could tune a “human style”. If not, I’d suggest Test Mode capture of your pairings as a small evaluation set.
- **N6 — Latency, cost and offline needs.** Is a sub-second decision acceptable? Must the game work when a Linux machine is down? My default: embedded model, so there is nothing to be down.
- **N7 — Do you already have a Linux machine you’d like to use?** Only relevant if option E is chosen.
- **N8 — What may the model see?** Only what a human would (technical spec Q-R11 / G3). Note that N13 is a variant of this question.
- **N9 — Appetite and timing.** Research side-project with no deadline, or something needed for a release?
- **N10 — Is keeping the weights server-side acceptable?** It requires putting the strategy outside the shared engine package (§6.3).

Appendix A – a minimal training sketch (illustrative, not run)

```
import torch, torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

# features.npz was exported by the TypeScript harness: X = [N, F] float32, y = [N, 1] expected
# value
X, y = load("features.npz")
F = X.shape[1]
train, val = split_by_scenario(X, y) # never split rows of the same scenario apart

model = nn.Sequential(nn.Linear(F, 128), nn.ReLU(),
                      nn.Linear(128, 64), nn.ReLU(),
                      nn.Linear(64, 1))
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.MSELoss()
loader = DataLoader(TensorDataset(*train), batch_size=512, shuffle=True) # keep num_workers=0
# on macOS

for epoch in range(30):
    for xb, yb in loader:
        opt.zero_grad()
        loss = loss_fn(model(xb), yb)
```

```

    loss.backward()
    opt.step()
    with torch.no_grad():
        print(epoch, float(loss_fn(model(val[0]), val[1])))
        # watch validation loss; stop when it stops improving

# 1) ONNX, for any runtime that wants it
torch.onnx.export(model, torch.zeros(1, F), "pairing_v1.onnx",
                  input_names=["x"], output_names=["score"], dynamic_axes={"x": {0: "batch"}})

# 2) plain weights for the TypeScript forward pass (option A)
weights = {name: p.detach().numpy().tolist() for name, p in model.state_dict().items()}

```

With F around 300 this model has roughly 47,000 parameters. A real run adds: a test set kept untouched, seeds, saving the best epoch, and the metadata described in §8.

Appendix B – a first-week checklist for Stage 2

1. Create the environment (uv or conda), install PyTorch, scikit-learn, LightGBM, NumPy, pandas, Jupyter.
2. Run the PyTorch “Learn the Basics” tutorial end to end once, so the loop in Appendix A is no longer mysterious.
3. Train a deliberately silly model (predict the average) to establish a baseline loss.
4. Train LightGBM on the exported rows; read its feature-importance output — it will tell you which inputs matter.
5. Train the MLP; plot training versus validation loss; learn what overfitting looks like by training too long.
6. Compare both against expert on held-out scenarios; write a one-page note with the numbers.

Sources

Convex (vendor documentation, checked 03-OCT-2026): - [Limits](#) - [Runtimes](#) - [Bundling \(external packages for Node actions\)](#) - [Actions](#)

Training and tooling: - [Introducing Accelerated PyTorch Training on Mac](#) - [Accelerated PyTorch training on Mac — Hugging Face Accelerate](#) - [MLX — Apple Open Source](#) - [Get started with MLX for Apple silicon — WWDC25](#) - [Export a PyTorch model to ONNX — ONNX Runtime](#) - [ONNX Runtime JavaScript \(node, web\)](#) - [Reducing the ONNX Runtime WebAssembly file size](#) - [m2cgen releases](#) - [OpenSpiel AlphaZero](#)

Figures I could not confirm from a primary source (the model-cost estimate in §6.3, effort estimates in §1.2 and §11, and licence summaries in §10) are marked as estimates or “confirm” in the text.